

Credit Card Fraud Detection System

End-to-End Machine Learning & MLOps Project Report

Project objective: Detect fraudulent credit-card transactions under extreme class imbalance while aligning the operating threshold with business cost.

Final model: LightGBM + SMOTE

Final threshold: 0.239

Metric	Result
PR-AUC	0.8744
ROC-AUC	0.9793
Precision	76.99%
Recall	88.78%
F1	82.46%
Financial Cost	136

This report documents the completed project from exploratory analysis through production-style inference, interpretability, experiment tracking, API validation, and automated testing.

1. Executive Summary

The project implements an end-to-end binary classification system using the public Kaggle Credit Card Fraud Detection dataset. Fraud represents only about 0.17% of transactions, making this a highly imbalanced learning problem. The project therefore treats accuracy as an inadequate primary metric and emphasizes PR-AUC, ROC-AUC, precision, recall, F1, and a business-cost function.

The workflow contains five stages: EDA and evaluation setup; leakage-safe preprocessing and feature engineering; model comparison and imbalance strategies; threshold tuning and SHAP interpretability; and production/MLOps integration.

- Candidate models: Logistic Regression, LightGBM, and XGBoost.
- Imbalance strategies: class weighting, SMOTE, and random undersampling.
- Threshold selection: based on out-of-fold predictions and asymmetric business cost.
- Production: serialized LightGBM + SMOTE pipeline, FastAPI, Pydantic validation, and pytest.

Final test-set results: 0.8744 PR-AUC, 0.9793 ROC-AUC, 76.99% precision, 88.78% recall, and 82.46% F1 at threshold 0.239. The project cost function produces a final cost of 136.

2. Problem Definition

Task: Predict whether a transaction is fraudulent (Class=1) or non-fraudulent (Class=0) using Time, Amount, and anonymized V1–V28 features.

Core challenge: extreme imbalance

The EDA quantifies the class distribution and demonstrates why an all-negative classifier can achieve very high accuracy while detecting zero fraud. This motivates precision-recall analysis as the central evaluation framework.

Business objective

The project assigns a cost of 10 to a false negative and 1 to a false positive. Missing fraud is therefore treated as substantially more costly than incorrectly flagging a legitimate transaction.

Component	Description
Class	Binary target: 0 non-fraud, 1 fraud
V1–V28	Anonymized principal-component variables
Time	Seconds elapsed since the first transaction
Amount	Continuous transaction amount with extreme values

3. Exploratory Data Analysis

Validation

The first notebook validates column names, data types, missing values, duplicate rows, descriptive statistics, and target distribution before modeling.

Amount analysis

Amount distributions are compared between fraud and non-fraud transactions, including class-wise descriptive statistics and skewness. Extreme values motivate RobustScaler in Stage 2, and transformations are not fitted on the full dataset.

Time analysis

Time is analyzed as seconds elapsed since the first transaction. Differences in temporal distributions motivate deriving hour-level and cyclical features.

PCA features

Because V1–V28 are anonymized principal components, the project does not assign business meaning to individual variables. It instead examines class-wise means, correlations with Class, and correlation structure.

Evaluation setup

The project establishes PR-AUC, ROC-AUC, precision, recall, F1, and financial cost before model selection. PR-AUC is especially important because it focuses on the rare positive class.

4. Preprocessing & Feature Engineering

Leakage-safe split

The train/test split is performed before transformations. This prevents test-set information from influencing scaling, feature statistics, or synthetic minority examples.

Robust scaling

RobustScaler is applied to Time and Amount. The choice is motivated by the extreme values observed in EDA.

Time features

Feature	Construction
Time_Hours	Time / 3600
Hour	Hour bucket derived from Time
Hour_Sin	$\sin(2\pi \times \text{Hour} / 24)$
Hour_Cos	$\cos(2\pi \times \text{Hour} / 24)$

Transaction-volume feature

Transactions_Per_Hour is calculated from training data using hourly transaction counts. Counts are mapped onto other splits, and the learned mapping is stored with the production artifact so inference reconstructs the same feature.

Pipeline design

The final training pipeline combines preprocessing, SMOTE, and LightGBM so that training-time transformations and resampling remain reproducible.

5. Model Comparison & Imbalance Handling

Stage 3 evaluates multiple model families rather than assuming one algorithm is best.

Approach	Role
Logistic Regression	Linear baseline
LightGBM	Nonlinear gradient-boosted tree model
XGBoost	Alternative boosted-tree model
Class weighting	Reweight minority-class errors
SMOTE	Synthetic minority oversampling during training
Random undersampling	Reduce majority-class representation

Stratified cross-validation compares approaches with PR-AUC. LightGBM + SMOTE produced the strongest PR-AUC among the evaluated candidates and was carried forward.

Why SMOTE

SMOTE increases representation of the minority class during training without modifying the untouched test set.

Why LightGBM

LightGBM + SMOTE provided the strongest PR-AUC among the evaluated candidate approaches and became the final production configuration.

6. Threshold Optimization & Business Cost

OOF selection

The final threshold is selected using out-of-fold predictions rather than direct optimization on the final test set. A vectorized threshold sweep evaluates the project cost function.

Quantity	Value
OOF PR-AUC	0.8559
Selected threshold	0.239
False-negative cost	10
False-positive cost	1

Why not 0.5?

A 0.5 probability cutoff is not automatically appropriate for fraud detection. The operating point depends on the relative cost of missed fraud and false alerts; the project therefore uses 0.239.

Interpretability

LightGBM global feature importance provides model-wide analysis, while SHAP supports global feature analysis and individual transaction explanations.

7. Final Model Performance

Metric	Test result
PR-AUC	0.8744
ROC-AUC	0.9793
Precision	76.99%
Recall	88.78%
F1 Score	82.46%
Threshold	0.239
Financial Cost	136

Confusion matrix

	Predicted Not Fraud	Predicted Fraud
Actual Not Fraud	56,838 (TN)	26 (FP)
Actual Fraud	11 (FN)	87 (TP)

At the selected threshold, the model identifies 87 of the 98 fraud cases in the test set and produces 26 false alerts. The test set was not used to fit the model or choose the threshold.

Production prediction example

```
Fraud probability: 0.000017
Threshold:        0.239
Prediction:       0
Decision:        NOT FRAUD
```

8. Production Architecture

```
Transaction payload
↓
FastAPI POST /predict
↓
Pydantic validation
↓
src/predict.py
  ■■ reconstruct time features
  ■■ apply learned hourly statistics
  ■■ load saved preprocessing + model
  ■■ apply stored threshold
↓
PredictionResponse
  ■■ fraud_probability
  ■■ threshold
  ■■ prediction
  ■■ decision
```

Serialized artifact

models/fraud_detection_lgbm_smote.joblib contains the trained ML pipeline, learned hourly transaction statistics, decision threshold, model metadata, and evaluation metrics.

API surface

Route	Purpose
GET /health	Service health check
POST /predict	Fraud probability and classification decision
/docs	Interactive API documentation

Input validation

Pydantic validation rejects missing required features, invalid numeric values, negative Amount values, and incorrect data types before the request reaches the model. Invalid requests return HTTP 422.

9. Automated Testing

The project includes pytest coverage for production-facing behavior and reusable preprocessing logic.

- FastAPI health endpoint.
- Missing API fields.
- Invalid transaction amounts.

- Time feature engineering.
- Transaction-volume feature engineering.
- Preprocessing.

The final suite reports **6 passed**. A Starlette/AnyIO deprecation warning was observed but did not cause test failure.

Engineering value

The tests establish regression checks around the API contract and feature transformations, reducing the risk that later changes silently break inference.

10. MLflow & Reproducibility

MLflow tracks experiments under the experiment name **credit-card-fraud-detection**. The project uses an SQLite tracking database, mlflow.db, rather than filesystem tracking.

```
MLFLOW_DB_PATH = PROJECT_ROOT / "mlflow.db"
mlflow.set_tracking_uri(
    f"sqlite:///{{MLFLOW_DB_PATH.as_posix()}}"
)
mlflow.set_experiment("credit-card-fraud-detection")
```

Reproducibility

The primary train/test split and cross-validation procedures use random_state = 42. The production artifact preserves learned training statistics, threshold, model metadata, and metrics required for inference.

Version-control discipline

The dataset, MLflow local database, Python caches, pytest cache, notebook checkpoints, and virtual environments are excluded from Git. The trained model artifact is retained.

11. Repository Structure

```
fraud-detection/
├── README.md
├── requirements.txt
├── data/creditcard.csv
├── notebooks/
│   ├── 01_eda_and_baseline.ipynb
│   ├── 02_preprocessing_and_feature_engineering.ipynb
│   ├── 03_model_comparison_and_imbalance.ipynb
│   ├── 04_threshold_tuning_and_shap.ipynb
│   └── 05_mlflow_experiments.ipynb
├── src/
│   ├── features.py
│   ├── preprocessing.py
│   ├── train.py
│   └── predict.py
├── api/
│   ├── main.py
│   └── schemas.py
├── models/
│   └── fraud_detection_lgbm_smote.joblib
└── tests/
    ├── test_preprocessing.py
    └── test_api.py
```

Separation of concerns

Component	Responsibility
notebooks/	EDA and experiments
src/features.py	Reusable feature engineering
src/preprocessing.py	Reusable preprocessing

Component	Responsibility
src/train.py	Training and artifact creation
src/predict.py	Production inference
api/	FastAPI interface and schemas
tests/	Automated validation
models/	Trained model artifact

12. Reproduction & Usage

Install

```
pip install -r requirements.txt
```

Dataset

```
data/creditcard.csv
```

Train

```
python src/train.py
```

Run prediction

```
python -m src.predict
```

Run API

```
python -m uvicorn api.main:app --reload
```

Run tests

```
python -m pytest
```

Notebook order: EDA → preprocessing/feature engineering → model comparison → threshold tuning/SHAP → MLflow experiments.

13. Limitations & Future Work

The project uses a public historical dataset and does not represent a production banking environment.

- Real-time transaction streams and latency requirements.
- Concept drift and data drift monitoring.
- Model monitoring and automated retraining.
- Fraud-investigation feedback loops.
- Business-specific cost calibration.
- Authentication and authorization.
- Privacy and regulatory requirements.
- Production observability.

Potential extensions

- Real-time streaming inference.
- Model and data drift detection.
- Automated retraining.

- Probability calibration.
- Cloud deployment.
- API authentication.
- CI/CD and advanced monitoring.

14. Final Technical Assessment

Capability	Evidence
EDA	Class imbalance, distributions, correlations, evaluation design
Modeling	Logistic Regression, LightGBM, XGBoost
Imbalanced learning	Class weighting, SMOTE, undersampling
Validation	Stratified CV and OOF predictions
Feature engineering	Cyclical time and transaction-volume features
Business alignment	Asymmetric cost and threshold optimization
Interpretability	Global importance and SHAP
MLOps	MLflow tracking
Production ML	Serialized inference pipeline
API	FastAPI + Pydantic
Testing	6 passing pytest tests

End-to-end workflow

EDA → Feature Engineering → Preprocessing → Imbalance Handling → Model Comparison → Cross-Validation → Threshold Optimization → Business Cost Evaluation → SHAP → MLflow → Model Serialization → FastAPI → Pydantic Validation → Automated Testing

Final result

The final LightGBM + SMOTE model achieves 0.8744 PR-AUC, 0.9793 ROC-AUC, 76.99% precision, 88.78% recall, 82.46% F1, and financial cost 136 at threshold 0.239 on the untouched test set.

The strongest engineering aspect is the continuity between experimentation and deployment: imbalance handling is evaluated, the operating threshold is tied to a stated business objective, interpretability is included, and the selected model is carried into a validated API.